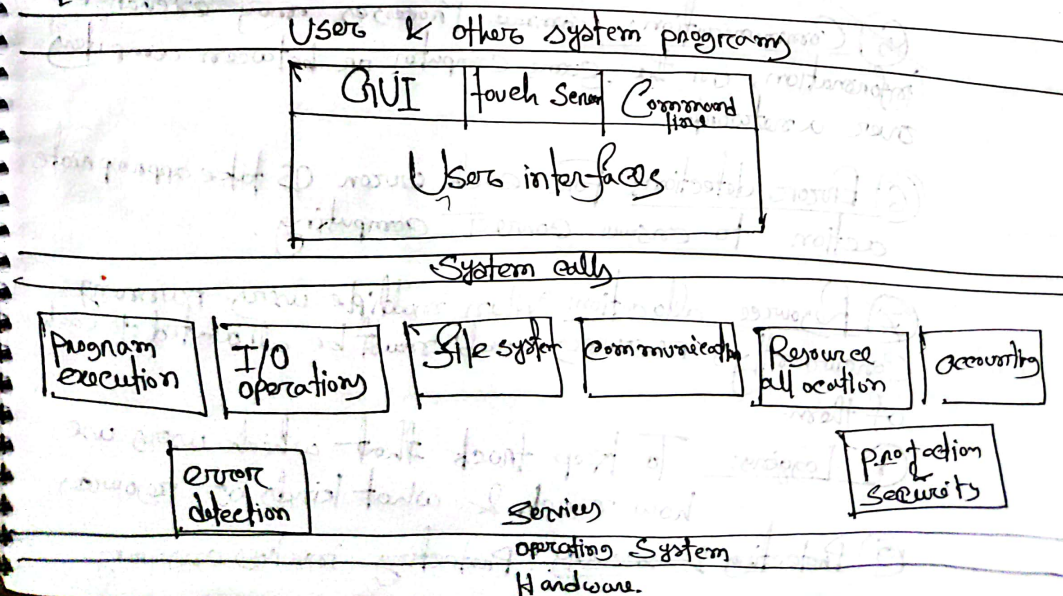


Chapter-2

Objectives

- * Identify services provided by an operating system.
- * Illustrate how system call are used to provide operating system services.
- * Compare & contrast monolithic, layered, microkernel, modular & hybrid strategies for designing OS.
- * Illustrate the process for booting an operating system.
- * Design & implement kernel module for interacting with a linux kernel.

① Operating system services.



OS: Operating system provides an environment for execution of programs & services to programs & users.

OS services: ① User interface: Almost all OS have a user interface (UI) variety between CLI (Command line interface), Graphics User interface (GUI), touch-screen.

② Program Execution: System able to load a program into memory & to run that program, and execution, either normally or abnormally.

③ I/O operations: A running program may require I/O which involve a file or an I/O device.

④ File system manipulation: Programs need to read & write files & directories, create & delete them, search them.

⑤ Communication: Comm. Processes may exchange information, on the same computer or between computers over a network.

⑥ Errors detection: For each error OS take appropriate action to ensure correct computing.

⑦ Resource allocation: When multiple users running concurrently, resources must be allocated to each of them.

⑧ Logging: To keep track that which users use how much & what kinds of resources.

⑨ Protecting & security: Protecting involves ensuring

Q.4
all access to system resource is controlled.

Security: security of system from outsiders requires users authentication

Booting Operation Process:

① Building the operating system: ②

Step 1: Write the source code; ^{write} Linux kernel source code can be obtained from kernel.org

Step 2: Configure the kernel: Use tools like make menuconfig to set up the OS configuration according to hardware.

Step 3: Compile the kernel: Compile the OS kernel using make command. this converts source code to machine readable format

Step 4: Compile & install kernel module: If the OS uses additional kernel modules, compile them using make modules.

Step 5: Install the kernel: Use make install command to install the newly compiled kernel onto system

Booting the OS:

Step 1: Power on & start bootstrap process: When a system powers on, the CPU begins executing instructions from fixed memory location.

This is where the bootstrap load is stored, in BIOS, UEFI

Step 2: Locate the OS: Bootstrap loaders identify & load the kernel image from disk into memory.

Step 3: Use the Boot loader: (GRUB)

GRUB is a command tool that allows users to select which OS or kernel version to boot.

Step 4: Load & execute the kernel:

The kernel image is loaded into memory & executed.

Monolithic kernel structure: - Original UNIX

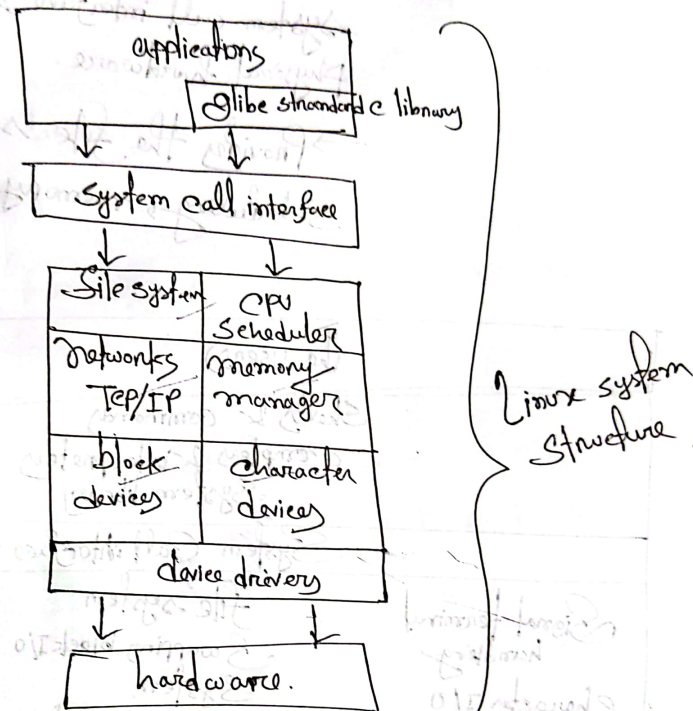


Fig: Monolithic plus modular design

UNIX: - limited by hardware functionality, the original Unix operating system had limited structure.

UNIX-OS consists of two parts:
 → System program.
 → kernel

~~The UNIX~~ The kernel

→ consist of everything below the system-call interface & above the physical hardware.

→ Provides the file system, CPU scheduling, memory management

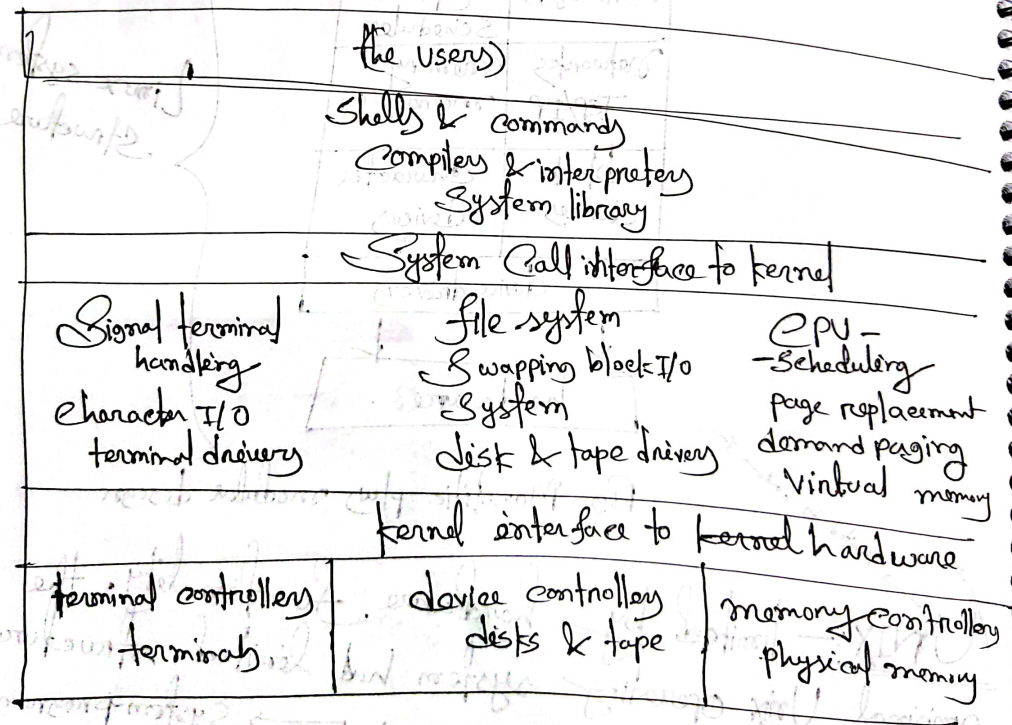


Fig: Traditional Unix system structure.

Hybrid system: Hybrid combines multiple approaches to address performance, security, usability needs.

It is combination of different models (such as monolithic, microkernel & layered) to achieve balance between performance, security, flexibility.

- Advantages:
- ① Flexibility allowing for ^{tailored} solution for specific needs.
 - ② Can optimize the system for both small & devices & large servers.
 - ③ Supports dynamic loading & unloading of modules.
 - ④ Balance speed of monolithic designs.

- Disadv:
- ① Combining different models increases the system's complexity.
 - ② Require more system resources.
 - ③ Resolve issues makes it challenging.

tailored
optimize
small &
large

TOOB
1/12
5

1,2,4,5

Design & implement Linux kernel Module:

(programming type)

① Create C file with basic functions

② Create Makefile;

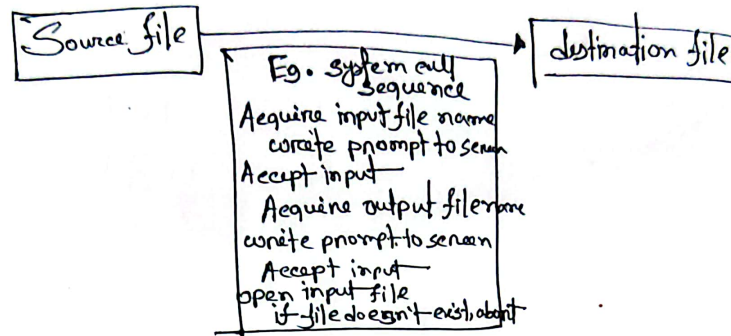
3. Compile using make.

4. Load the module: Sudo insmod simple-module.ko

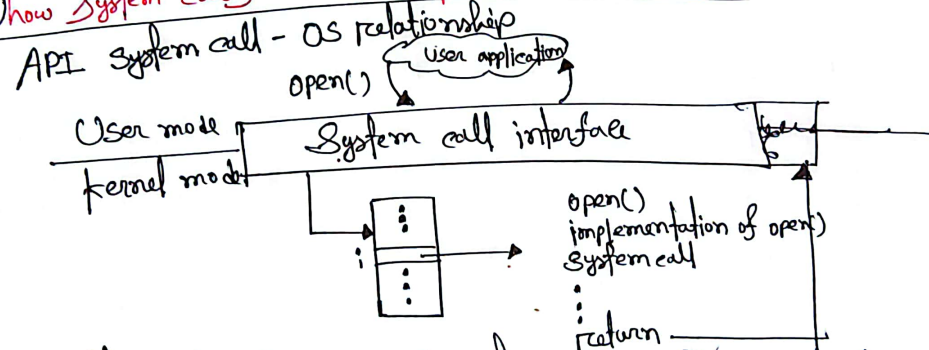
5. Unload the module: sudo rmmod simple-module.ko

6. Check logs: cat dmesg | tail for messages.

example of system calls:



② how system calls are used to provide OS services:



System call invokes the intended system call in OS kernel & return status of system call.

The caller needs to obey API & understand what OS will do as a result call.

Chapter-2

Mahbub Sir..

System call: ^{সিস্টেম} Only users application or operation perform ^{সিস্টেম} with higher privileges (hardware access, memory recovery or interact each other) require ^{সিস্টেম} to perform operation directly execute ^{সিস্টেম} with ^{সিস্টেম} security & stability reason as ^{সিস্টেম} ^{সিস্টেম} system call ^{সিস্টেম} (to ask the kernel to perform the operation on its behalf).
(instead, it uses)

Purpose of a system call: ~~RFPi~~ (RFPi)

① Resource management: allocation & deallocation of resource handle ^{সিস্টেম}. Eg: malloc (memory allocation)
free (memory deallocation)

② file management: They allow application to open, close, read, write & delete files. Eg: open(), read(), write(), close().

③ Process control: manage processes, including creation, termination eg: fork(), exec(), exit().

④ inter process communication: enable communication & data exchange between different processes.

Eg: pipe(), shmget() (shared memory),
msgsnd() (message passing).

Command interpreter:

Command The command interpreter often called shell, users interface provide ~~an~~ operating system ~~an~~ user interact ~~with~~ ~~the~~ ~~OS~~. The shell can be either a text-based-command line interface (CLI) or a graphical user interface (GUI).

Purpose of command interpreter:

① accepts command from the user.

↓
execute & interpret them

↓
execute the corresponding system functions

2, 3, 4, 5
② Error handling: for invalid input, it provides feedback to users to guide them.

③ provides a way for user to interact with OS, issue commands & receive output.

Why is the command interpreter separate from the kernel?

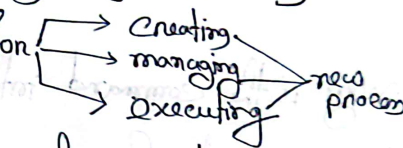
Ans: ① kernel focuses on low level tasks like managing hardware, memory processes, & system security. While the command interpreter is a high level program focused on user interaction.

② Since shell is separate, user commands are can replace it easily. For eg. on unixlike system, users can choose between different shells like Bash, Zsh or Fish.

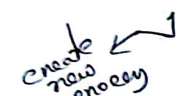
Each shells offers different features, but all interact with kernel in constant way.

(23) What system calls have to be executed by a command interpreter or shell in order to start a new process on a UNIX system?

Ans: To start a new process on a UNIX system, the shell executes a sequence of system calls.

These calls are responsible for 

When shell starts a new process to execute command, the workflow is:

1. fork(): The shell uses fork() to create a new process. The child process is an exact copy of new parent.

2. exec(): The child then uses one of exec() system calls to replace its image with program need to be executed.

3. wait(): The parent process waits for child process to finish by calling wait(), retrieve the child exit status.

4. exit(): The child process really exit() when it finishes.

For eg:

when run the ls command in a shell, the shell will

1. Use `fork()` to create child process
2. In child process, use `execvp()` to replace child's memory space with ls program.
3. The parent process will use `wait()` for the ls process to finish.
4. After ls process complete, it'll call `exit()` & return status to parent

Purpose of system program:

System program is software, it essential services & functionalities between user, hardware & application software on a system to manage & control user.

Purpose: ① File management: Program like cp, rm, mv, ls help users to manage file.

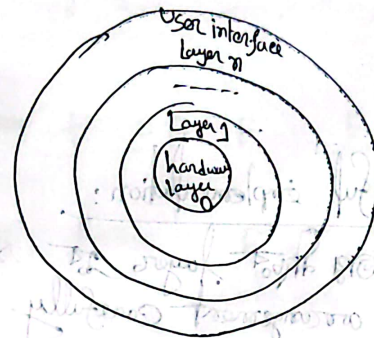
2. Process management: utilities such as ps, kill & top monitors & control running process.
 3. Device Management: device programs allow users to manage hardware devices.
 4. Info. sharing: Tools like mail, remote enable communication between users.
-

advantage & ~~dis~~ disadvantages of layered approach:

Layered approach: System structure present as a diff. services split into layer a split up. Present into layer specific well-defined task perform.

It is created to improve pre existing structured simple structure.

Thus if users layer wants to interact with the hardware layers, the response will be ~~then~~ traveled through all the layers from $n-1$ to 1 .



Advantage:

① ~~यह संरचना को आसानी से समझा जा सकता है, और इसे~~

(This design promotes modularity as each layer performs only the task, it is scheduled to perform.

② As each of the layers are discrete so it is easy to debug.

③ If modification is made in particular layer, it won't effect other layers.

No direct access to hardware.

① ~~A users~~

A user can use the services of hardware but can't directly modify it.

Disadvantage:

① Complex & Careful implementation:

~~If~~ If a layer, or a set of layers, get service from other layers, the layers arrangement carefully must be done.

A layer receives services of the layers below it, so implementation must be done carefully.

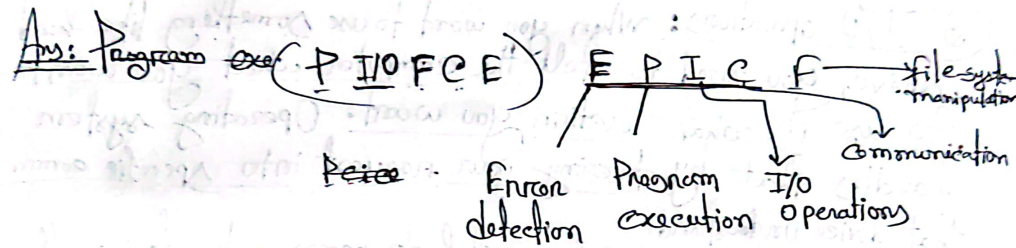
Eg: Backing storage layer uses the service of the memory management layer. So it must (b.s) be kept below the memory management layer.

② Slower in execution:

If a layer wants to interact with other layers, it sends a request that has to travel through all the layers between two interacting layers.

Thus it increases response time.

✓ List five services by OS provided by OS, explain how each creates convenience for users. In which cases would it be



① Error detection: hardware level (like wires, chips or disks)
↳ data read move etc (like memory to processor), computer check if sure that data damage or changed that.
↳ data write disk & data save etc, computer checks if data damage or change etc accidentally.

Software level: (the programs that run on the computer): Data on storage still makes sense.

Operating system handles any errors that are found so other programs don't have to worry about finding every possible mistake.

② Program execution: OS file ~~me~~
operating system file contents memory to load
→ ~~OS~~ & execution ~~of~~ ~~OS~~; [OS loads contents of
A user level program could not be trusted to allocate CPU time. a file into memory & begins execution]

③ I/O operations: When you want to use something like hard drive, you need to tell the computer: what you want to use & what action you want. Operating system handles that by turning your request into specific commands that device understands.

User level program cannot be trusted to access only devices they should have access to & access them when they're ~~are~~ unused.

④ file-system manipulation: When we create, delete or use files on computer, there are many behind-the-scenes tasks that happen, like file-creation, file deletion, file access, etc. If users tried to handle these tasks themselves, mistakes could happen. OS takes care of all these details to run everything smoothly.

⑤ Communication: Message passes between system
→ frequency message turned into packets
→ Sent to the network controller.
→ transmit across a communication medium.

User programs might receive packets destined for other process.

Q.7 Why do some systems store the OS in firmware, while others store it on disks?

Ans: Systems store the OS in firmware:

- ① Firmware is a type of permanent memory built into devices, doesn't require hard disk.
- ② ~~Embedded~~ Embedded Systems (microwave, tv) store the OS in firmware because these systems do specific tasks & don't need changeable OS.
- ③ OS starts ~~immediately~~ immediately when system ~~turn on~~ turns on since it

Systems store the OS on disk:

- ① Computers with complex, full featured operating systems (Windows, Linux, macOS) store OS on hard disk because OS is much larger &

updated.

② Disk storage allows for bigger & more flexible operating system that can be changed, updated or customised easily.

②-8 How could a system be designed to allow a choice of ^{operating} program systems from which to boot? What would be the bootstrap program need to do?

Ans: To design a system that allows a choice of OS to boot from, we need to implement a boot manager. Consider a system that would like to run both window & three different dist of Linux. Each operating system will be stored on disk.

It works in simple terms:

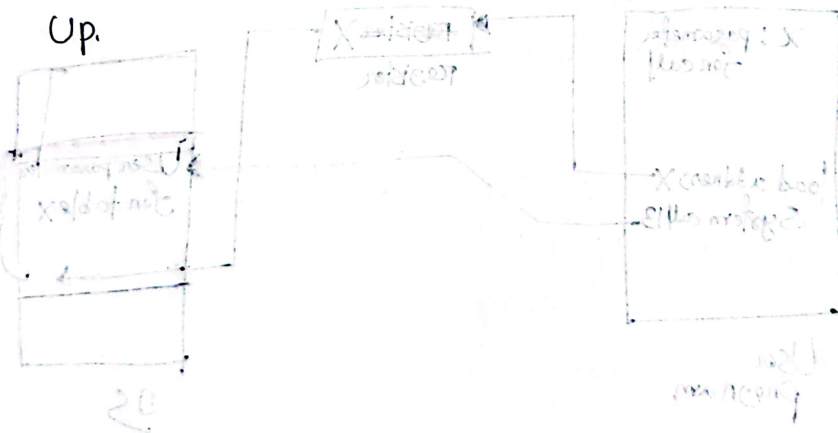
1. When you turn on your computer, instead

of going straight into an OS, the boot manager runs first.

2. Boot manager lets you choose which OS you want to use. like windows or one of Linux version.

3. If you don't make choice, boot manager will automatically start a default OS after a short wait.

4. To work properly, the boot manager must be stored in specified location on the hard drive, so computer knows where to find it when starting up.

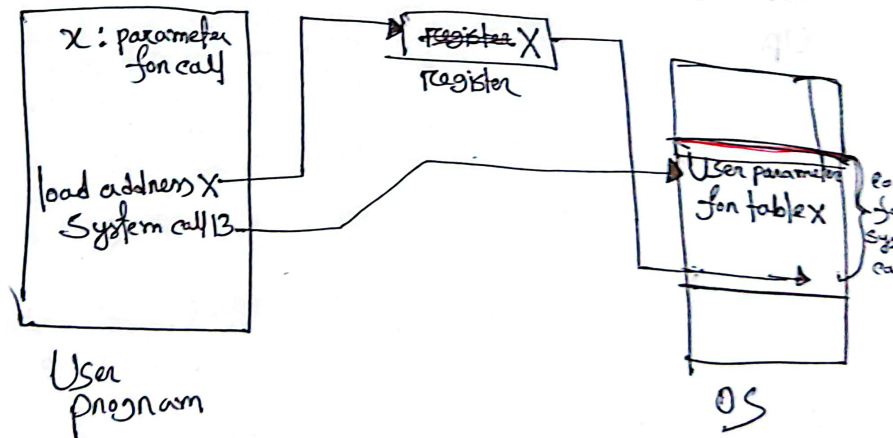


^{system call}
Q.10) Describe three general methods for passing parameters to the OS.

Ans: ① Pass parameters in register

② Register pass starting address of block of parameter

③ Program can be placed, or pushed onto the stack by the program. & popped off the stack by OS.



Q.9 The services & functions provided by an OS can be divided into two main categories. Briefly describe the two categories & discuss how they differ.

Ans: Two main classes of services:

1. Protection services: Different programs running at the same time but don't ~~interfere~~ interfere with each other.

a. memory protection: Each process is given its own separate area of memory to use. This prevents one process from accessing another process's memory.

b. file protection: Processes can only access files they're permitted to use. This prevents one user's files from being changed by another user's process.

③ Device protection: Processes can't directly access hardware devices without OS.

These services protect the system's resources.

2. Functionality Services:

These services ^{add} functions that computer's hardware doesn't provide by itself.

like : Virtual memory.
File system.

Protection services safeguard system resources.
Functionality services extend the capabilities of the hardware to make system more powerful & user-friendly.

2.11 Describe how you could obtain a statistical profile of the amount of time a program spends executing different sections of its code. Discuss the importance of obtaining such a statistical profile.

Ans: ① The idea is to use periodic timer ~~in~~ interrupts to take snapshots of the program's state at regular intervals.

② When each interval occurs, we check which part of code is currently executing.

③ By gathering many samples, you create a statistical profile showing which sections are active most, which corresponds to where the program spends most of its time.

④ If a specific section appears frequently in snapshots, it indicates that this part of the code

is consuming more CPU resource.

⑤ With this information, programmer can focus on optimizing these high-usage sections to improve program performance.

Importance of obtaining such a statistical profile:

1. Focus on optimizing efforts.
2. Avoid personal premature optimization.
3. Improve program efficiency.
4. Debugging behavior (how program behaves during execution).

Q.12 Advantage & disadvantage of using same system
- call interface for manipulating both files & devices?

Ans: Advantage: ①

① Uniformity & simplicity: Using same system-call interface simplifies programming. Developers use read(), write, open() operation for both files & devices for making it easier to write & maintain code.

② Code reuse/reusability: Since the same functions are used, existing file code for file handling can often be reused for devices with minimal changes.

③ Modularity: It allows files & devices to be treated as streams of bytes, promoting modularity.

④ Consistency in error handling.

disadvantage:

① Limited functionality:

② Performance overhead.

③ Complex error handling.

Q.13) could it be possible for users to develop a new command interpreter using the system the system-call interface provided by the operating system?

Ans: The command interpreter allows a user to create & manage processes & also determine ways by which they communicate. As all of these functionality could be accessed by a user level program files using the system calls, so it is possible.

Q.14) Describe why Android uses ahead of time (AOT) rather than just-in-time (JIT) compilation.

Ans: Android uses Ahead-of time (AOT) compilation to make apps run faster & more efficient.

① In most Java environments, when you run an app, the code is compiled while it is running (Just in time). This means the app might be slower at first & use more CPU & more power.

② With AOT, android ~~also~~ compiles app into machine code when we install it on our phone instead of waiting until we run it. Making it much quicker to launch.

So. The benefits are: ① Faster performance.
(AOT)

② Less battery use

③ Lower memory usage.

Q.15 What are two models of inter-process communication? What are the strength & weakness of two approaches?

Ans: Two models are:

① Message passing: In this model, processes communicate by sending messages to each other. Each process has its own memory space & sends data as discrete message. Use in ^{computer} inter-process communication. _{Small data exchange}

② Shared memory space model: In this model, multiple processes access a common memory space. This allows them to read & write data directly, making communication faster & more efficient.

Use case → high speed communication
 → large data transfer.

Strength of message passing model: ~~ADDE~~ ~~PD~~ ~~EDD~~

- ① Simplicity: easier to implement, understand & process are decoupled.
- ② No data conflict: as process don't share memory, so no risk of data corruption.
- ③ Flexibility: Can handle dynamic process creation & communication easily.
- ④ Error handling: error in one process don't directly affect others.

Weakness of message passing ~~(SLE)~~

- ① Performance overhead: Communication can be slower due to message passing & handling.
- ② Limited data transfer.
- ③ Complexity in communication.

Strength of memory model:

- ① High speed: Effi.
- ② Low Latency: Direct data access to shared data leads to minimal communication delay.
- ③ Resource Efficiency: Reduces the overhead associated with message handling.

Weakness:

- ① Complexity in synchronization
- ② Protection issues
- ③ Limited to local communication.

2.16 ~~Contrast~~ Contrast & compare an application programming interface (API) & an application binary interface (ABI)

2.17 X

2.18 It is so sometimes difficult to achieve a layered approach if two components of the OS are dependent on each other. Identify a scenario in which it is unclear how to layer two system components that require tight coupling of their functionalities.

Ans: The virtual memory subsystem & ~~storage~~ storage subsystem are tightly linked or coupled because they need to co-ordinate how they manage & storage resources.

① file mapping into virtual memory:
⇒ many OS allow files stored in storage system to be directly mapped onto a process's virtual memory space.

⇒ This means when a process access a part of VM it actually actually reads data directly from storage device.

⇒ Because of this, this VM sys. subsystem interact with storage subsystem to retrieve the necessary file data, creating a dependency between the two.

② VM using storage as a backing store:

⇒ VM can often use storage as a backing store for pages that aren't in physical memory.

⇒ When the system needs to swap data in & out of physical memory, it relies on storage subsystem to read from or write to the disk.

diff 3. Buffered file system updates:

— The storage subsystem may use physical memory to temporarily buffer file updates, before writing them to disk.

⇒ Since buffered data uses physical memory, it must be managed to avoid conflicts with VM sys.

Subsystem, which also needs memory for its own operation.

It is difficult to separate layers because

(a) VM system relies on storage system to handle handle files & swap space.

(b) Storage system relies on VM system to allocate & manage physical memory for buffering.

Q19) What is the main adv. of microkernel approach to system design?

Ans: (a) Improved stability & reliability:

- ① Easier to extend microkernel
- ② Easier to port the OS to new architectures
- ③ More reliable (key code running)
- ④ More secure
- ⑤ Reduce chance of a complete system crash. if one crashes, the core system (kernel) remains unaffected.
- ⑥ Enhance Security: With fewer components running kernel mode, attack surface is smaller, making it harder for malicious program to compromise the kernel.

(c) Ease of maintenance & updates: Simplifies bug fixes & feature additions.

④ Modular Modularity.

⑤ Fault isolation: Make system more robust

2.18 Disadvantage of microkernel approach:

① Performance overhead: Context switching between user mode & kernel mode is more frequent, leading to slower system performance.

② Complexity in design & implementation:

③ Increased communication cost.

④ Potential for lower throughput.

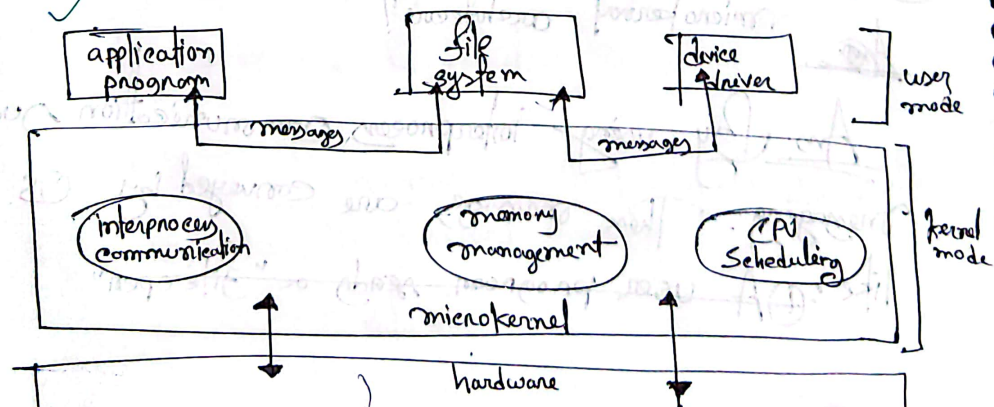
⑤ Larger memory footprint.

2.19 How do user programs & system services interact in microkernel architecture?

Ans. By using interprocess communication such as messaging. These messages are conveyed by OS.
like: ① A user program sends a "file open"

For. ex: opening a file:

1. A user program sends a "file open" message to the file system service.
2. The microkernel routes the message to the file system service.
3. The file system service opens the file, sends back a file handle to the user program via another message.
4. The microkernel then delivers this message to the user program.



Q.20 Adv. of using loadable kernel modules?

Ans: A loadable kernel module is a piece of code that can be dynamically loaded into the kernel at runtime to extend its functionality.

Adv.: ① Dynamic extension & modification of kernel functionality without rebooting.

② Simplified maintenance, customization & debugging capabilities.

③ Reducing memory footprint.

Q.2) How OS & android similar? How they differ?

Similar:

- ① Both based on existing kernel
- ② Both have an arc. that uses software stacks
- ③ Both provide frameworks for developers.

Difference:

- ① IOS is closed source, android open source.
- ② ios applications developed in objective C, android in Java.
- ③ Android uses VM, ios executes code natively.

Q.22 Explain why Java program running on android system & do not use standard Java API & VM.

Ans: Because standard API & VM are designed for desktop & server systems not mobile devices. Google developed a separate API & Virtual machine for mobile devices.

Q.23 Explain the architecture of Android.

Android architecture is based on Linux kernel. It consists of several layers. The bottom layer is the Linux kernel, which provides the hardware abstraction. Above the kernel is the Android Runtime (ART) layer, which contains the Dalvik VM (now replaced by ART) and the Android Framework. The top layer is the Android Application layer, which contains the user applications.

Android architecture is based on Linux kernel. It consists of several layers. The bottom layer is the Linux kernel, which provides the hardware abstraction. Above the kernel is the Android Runtime (ART) layer, which contains the Dalvik VM (now replaced by ART) and the Android Framework. The top layer is the Android Application layer, which contains the user applications.

Q.13 In what ways is the modular kernel approach similar to the layered approach? In what ways does it differ from the layered approach?

Ans: Similarity: ① Structured Design & separation of Concerns:

⇒ Both divide kernel into separate subsystems, making it easier to manage.

⇒ each subsystem has a well defined interface, similar to building blocks.

② Narrow interfaces: ⇒ in both models, subsystems expose a limited & well defined set of functions to interact with other subsystem.

⇒ helps reducing complexity & ensuring that only necessary functionality is exposed.

⇒ improve security & stability.

Difference: ① Freedom of interaction: layered kernel communicates in a strict top-to-bottom order.

Lower kernel cannot access or call functions in upper layers.

modular kernel: modular parts are not restricted by order & can freely interact with others in any direction.

② Flexibility: Layered kernel follows a more rigid structure.

modular kernel offers more flexibility since modules can interact dynamically.

③ Layered kernel is like a stack of layers in cake. where each layer can interact with the one below it.

modular kernel is more like a network of interconnected nodes where any part can talk to any other parts.